

AI AGENT 学习路线 · 学习专题 005

RAG 检索增强 学习笔记（打印版）

11 个篇章 · 7 张矢量架构图

生成日期：2026-09-19 | 由 Markdown 笔记自动排版生成

目录

01	005 - RAG 检索增强：专题导航	导航
02	00-背景篇：RAG 前史与全景	含架构图 ✓
03	技法篇导航 (techniques\)	
04	01-问题篇：为什么长上下文杀不死 RAG	
05	02-概念篇：核心概念精讲 (12 张概念卡)	含架构图 ✓
06	03-原理篇：检索管道解剖	含架构图 ✓
07	04-进阶篇：高级检索与 Agentic RAG	含架构图 ✓
08	05-实战篇：从零搭建贸易知识库问答 (Python 全代码)	含架构图 ✓
09	06-生产篇：评估、索引维护与成本	含架构图 ✓
10	07-对比篇：方案选型与向量库对比	
11	08-自测篇：题库与面试题	分级题库 ✓

共 11 个篇章。

005 - RAG 检索增强：专题导航

【本页问题】① 这个专题讲什么、按什么顺序学？② 文件怎么组织、图在哪？③ 和 001/002/003/008 专题怎么衔接？④ 贯穿案例是什么、我作为 CTRM 开发者能带走什么？

专题定位

RAG (Retrieval-Augmented Generation, 检索增强生成) 是所有企业级 LLM 应用中**工程密度最高、回报最确定**的一块：不改模型权重，靠"外挂知识库 + 检索"解决大模型的知识陈旧、私有知识缺失、幻觉三大痛点。

本专题以 **CTRM 贸易知识库问答** 为贯穿案例：把合同模板、业务规则、操作手册、历史点价案例四类资产建成知识库，回答"点价流程是什么""升贴水怎么算"这类问题。你不需要换技术栈——文中的向量库、权限过滤、评估等概念都可以映射回你熟悉的 Java 11 + Vue 2.7 + MySQL 工程经验。

目录树

005-rag检索增强\	
├ README.md	← 本页：导航与衔接
├ 00-背景篇-RAG前史与全景.md	← RAG 从哪来、解决什么、全景地图
├ techniques\	
│ └ README.md	← 技法篇导航与阅读顺序
│ └ 01-问题篇-为什么长上下文杀不死RAG.md	
│ └ 02-概念篇-核心概念精讲.md	← 12 张概念卡（费曼式）
│ └ 03-原理篇-检索管道解剖.md	← 索引管道 + 查询管道逐环节拆解
│ └ 04-进阶篇-高级检索与AgenticRAG.md	
│ └ 05-实战篇-从零搭建贸易知识库问答.md	← Python 全代码，可跑
│ └ 06-生产篇-评估索引维护与成本.md	
│ └ 07-对比篇-方案选型与向量库对比.md	
│ └ 08-自测篇-题库与面试题.md	
└ diagrams\	← 7 张 SVG 图册
├ 005-01-RAG技术演进时间线.svg	
├ 005-02-RAG两管道解剖.svg	← 重点图
├ 005-03-RAG核心概念地图.svg	
├ 005-04-分块策略对比.svg	
├ 005-05-混合检索与RRF原理.svg	
├ 005-06-贸易知识库问答架构.svg	
└ 005-07-RAG评估与失败模式对照.svg	

推荐学习路径

阶段	读什么	你将获得
1 建立全景	00 背景篇	RAG 的前史与 2026 现状，知道"为什么是它"
2 回答质疑	01 问题篇	面对领导"有了长上下文还要 RAG 吗"能给出成本/新鲜度/权限三连反驳
3 概念扫盲	02 概念篇	12 个核心概念的【一句话人话】+ 【生活类比】

阶段	读什么	你将获得
4 深入原理	03 原理篇	两条管道逐环节的职责与失败模式
5 进阶视野	04 进阶篇	父子块、GraphRAG、Agentic RAG、长上下文裁定
6 动手落地	05 实战篇	跑通一个 CTRM 知识库问答 (Python 全代码)
7 生产就绪	06 生产篇	RAGAS 评估、索引维护、权限过滤、成本优化
8 选型决策	07 对比篇	向量库/embedding 模型选型表、三方案决策表
9 查漏补缺	08 自测篇	题库 + 面试题自检

与其他专题的衔接

衔接点	说明
001 (LLM 基础)	RAG 的"G"端是 LLM：上下文窗口、token 计费、temperature 这些概念在 001 铺垫；本专题的 context packing 就是在为 001 讲的窗口预算做工程化
002 (Prompt 工程)	RAG 的"引用接地"本质是 prompt 模板设计：答案必须标注 [来源 n]、答不上要拒答，这套模板范式在 002 展开
003 (Agent 架构)	最强关联： 04 进阶篇的 Agentic RAG 把"检索"降级为 agent 的一个工具，由 agent 在 ReAct 循环中决定何时查、查什么、查几轮——工具循环协议见 003
008 (工程化/生态)	06 生产篇的评估体系、缓存、成本监控是通用工程话题，可带着 RAG 这一具体场景回到 008 的工程化框架中复用

贯穿案例：CTRM 贸易知识库问答

知识资产	示例问题	检索难点
合同模板	"背靠背合同的价格调整条款怎么写？"	条款编号与正文必须同块，切块不能切断
业务规则	"升贴水怎么算？"	规则文档常有表格，需要清洗与结构保留
操作手册	"点价流程是什么？"	步骤类长文，适合父子块（小块检索大块作答）
历史点价案例	"2025 年三季度铜点价有没有在贴水扩大时锁价的案例？"	时效性 + 数值精确匹配 → BM25 强项，混合检索价值大

图册速查

图	用在哪	一句话
005-01	00 背景篇	2020 论文 → 2023 工程化 → 2024 长上下文冲击 → 2026 Agentic/GraphRAG
005-02	03 原理篇	索引管道 + 查询管道全景，标注每步失败模式（重点图）

图	用在哪	一句话
005-03	02 概念篇	12 个核心概念分四组的关系地图
005-04	03 原理篇	固定/递归/语义/父子块四种分块策略对比
005-05	03 原理篇	BM25+向量双路召回、RRF 融合、rerank 收口的原理
005-06	05 实战篇	CTRM 知识库问答端到端架构
005-07	06 生产篇	RAGAS 四指标与失败模式的对照诊断

本专题事实性内容（模型版本、价格、GraphRAG 进展）已用 WebSearch 按 2026-09 时点核实并在正文标注来源；查不到权威披露的信息均显式标注。

00-背景篇：RAG 前史与全景

【本节问题】① RAG 是什么时候、由谁提出来的？② 在 RAG 之前人们怎么做机器问答？③ 2023~2026 这几年 RAG 经历了哪些阶段？④ 到 2026 年的今天，RAG 在企业 AI 版图里处于什么位置？

1. RAG 的前史：它站在信息检索的肩膀上

RAG 不是凭空发明的。它 = 信息检索 (IR) 传统 + 神经生成模型 的合流：

年代	技术	遗产
1970s~	布尔检索、倒排索引	图书馆式的"关键词命中"范式，今天的 Elasticsearch 仍是它
1994	BM25 (Robertson 等提出概率检索框架)	至今仍是关键词检索的天花板，RAG 混合检索中的一路
2013	word2vec	"语义可以用向量表示"的起点
2018~2019	BERT、DPR 的前奏	双塔/交叉编码器架构成型
2020.03	DPR (Karpukhin et al.)	稠密段落检索证明向量检索在 QA 上超越 BM25
2020.05	RAG 论文 (Lewis et al., Facebook AI, NeurIPS 2020)	正式提出 Retrieval-Augmented Generation：把维基百科检索器接进 seq2seq 生成器

关键认知：RAG 论文里的 RAG 是一个**模型训练方法**（端到端微调检索器+生成器）；今天工程界说的 RAG 是**无训练的系统架构**（冻结的 LLM + 外挂检索）。一词两义，面试要能分清。

2. 2023：工程化元年

ChatGPT 引爆后，企业最急的三件事恰好都是 RAG 的靶子：

- 1. **知识陈旧**：模型训练截止日之后的事一概不知；
- 2. **私有知识缺失**：公司合同模板、业务规则不在任何公开语料里；
- 3. **幻觉**：一本正经编造，企业场景不可接受。

于是 RAG 从论文走向工程：LangChain、LlamaIndex 框架化，Milvus/Qdrant/Weaviate/pgvector 向量库爆发，"ChatDocs"类产品满天飞。这个阶段的典型形态是 **Naive RAG**：切块 → 向量化 → top-k 召回 → 塞进 prompt。

3. 2024：长上下文冲击与混合检索成熟

Gemini 1.5 把上下文推到百万 token，"把整个知识库塞进上下文"的声音四起，RAG 第一次遭遇生存质疑。但同年三件事巩固了 RAG：

- **微软 GraphRAG 发布**：用知识图谱 + 社区摘要解决"全局性问题"（原始 GraphRAG 索引成本极高，约 3.3 万美元量级/大数据集）；
- **混合检索成为标配**：BM25 + 向量双路召回 + RRF 融合 + cross-encoder 重排的两阶段范式定型；
- **"Lost in the Middle"**（Liu et al., 2023）结论被广泛引用：长上下文中间部分的信息利用率显著下降——长 ≠ 全记得住。

4. 2025~2026：从"系统"降维成"能力"

- **LazyGraphRAG**（微软，2025-06 发布）：把 LLM 摘要推迟到查询时执行，索引成本降到全量 GraphRAG 的约 0.1%，全局查询质量持平且查询成本低约 700 倍（来源：Microsoft Research Blog 及 2026 RAG 路线图综述，nemorize.com，检索于 2026-09）；微软 GraphRAG 库 2026-03 发布含性能优化的新版本，相关基准工作被 ICLR 2026 接收（来源：microsoft.github.io/graphrag）。
- **混合路由成为共识**：没有人再跑"纯 GraphRAG"或"纯向量 RAG"——查询分类器把简单事实题路由给向量 RAG、多跳关系题路由给 GraphRAG（来源：bestaiweb.ai 对比分析，检索于 2026-09）。
- **Agentic RAG**：检索不再是固定管线，而是 agent 的工具，何时查、查什么、查几轮由模型决策（详见 04 进阶篇，协议衔接 003 专题）。
- **模型侧**：embedding 与 rerank 都已"两阶段定型"——bi-encoder 召回 + cross-encoder 重排（详见 07 对比篇的模型选型表）。

RAG 技术演进时间线 (2020–2026)

从端到端训练方法，到 Agent 时代的可授权知识层



LazyGraphRAG 效果数据来源：Microsoft Research Blog（2025-06 发布，检索于 2026-09）

结论：RAG 没有死，它从「系统」降维成 Agent 手里一件趁手的「能力」

5. 全景：RAG 到底解决什么

一句话：**RAG 把"知识"从模型权重里外置出来，变成可管理、可更新、可授权、可引用的数据资产。**

四个"可"，对应企业四大刚需：

可	需求	CTRM 映射
可更新	知识新鲜度	2026 年新版点价规则上线，当天生效，不用等模型换代
可管理	内容治理	合同模板 v3.1 替换 v3.0，删旧索引即可
可授权	权限隔离	期货部与现货部各看各的规则库（多租户）
可引用	可追溯/合规	答案标注"出自《点价操作手册》第 3.2 节"，审计可查

以及一个反向认知：RAG **不能**解决的事——模型推理能力本身的问题（多步算术、复杂逻辑）、格式遵循、语气风格（那是微调的活）。详见 07 对比篇的"RAG vs 长上下文 vs 微调"决策表。

【本节自测】

1. **2020 年的 RAG 论文和今天工程说的 RAG 有什么区别？** 要点：前者是端到端训练方法（联合微调检索器与生成器）；今天是冻结 LLM + 外挂检索的系统架构，无需训练。
2. **BM25 是哪一年、什么框架下的产物？它在 2026 年的 RAG 里还有位置吗？** 要点：1994 年概率检索框架；有——混合检索中的关键词召回一路，专治编号、品名、数值等精确匹配。
3. **2024 年 RAG 遭遇的最大质疑是什么？** 要点：长上下文（百万 token）冲击——"直接塞进上下文不就行了"。
4. **GraphRAG 为什么最初普及慢、后来怎么破局的？** 要点：原始版索引要海量 LLM 调用（成本约 \$33K 量级/大数据集）；LazyGraphRAG（2025-06）把摘要推迟到查询时，索引成本降至约 0.1%（来源：Microsoft Research Blog）。
5. **用"四个可"总结 RAG 的企业价值，并各给一个 CTRM 例子。** 要点：可更新（新规则当天生效）、可管理（模板版本替换）、可授权（部门隔离）、可引用（答案标出处）。
6. **RAG 解决不了哪类问题？** 要点：模型自身推理/算力问题、风格与格式遵循——那是微调与 prompt 工程的地盘。

技法篇导航 (techniques\)

【本页问题】① 技法篇 8 个文件按什么顺序读？② 每篇解决什么问题？

阅读顺序与定位

序	文件	一句话定位
01	01-问题篇-为什么长上下文杀不死RAG.md	用成本/注意力/新鲜度/权限四张牌回应"RAG 已死论"
02	02-概念篇-核心概念精讲.md	12 张概念卡，每张【一句话人话】+【生活类比】
03	03-原理篇-检索管道解剖.md	索引管道与查询管道逐环节拆解 + 失败模式表
04	04-进阶篇-高级检索与AgenticRAG.md	父子块、GraphRAG、Agentic RAG、长上下文裁定
05	05-实战篇-从零搭建贸易知识库问答.md	Python 全代码：chromadb + BM25 + RRF + rerank
06	06-生产篇-评估索引维护与成本.md	RAGAS 四指标、索引更新、权限过滤、成本优化
07	07-对比篇-方案选型与向量库对比.md	向量库/embedding 选型表 + 三方案决策表
08	08-自测篇-题库与面试题.md	20 题题库 + 面试大题

两条阅读路线

- 赶时间落地：02 概念 → 03 原理 → 05 实战 → 06 生产，两天能出 demo；
- 面试准备：01 问题 → 02 概念 → 04 进阶 → 07 对比 → 08 自测，重理解与对比。

01-问题篇：为什么长上下文杀不死 RAG

【本节问题】① 长上下文到底进步到什么程度？② "把知识库全塞进去"为什么不划算？③ "Lost in the Middle" 说明了什么？④ 权限、新鲜度这两张牌为什么是 RAG 的护城河？

1. 对手的实力：长上下文确实很能打

时点	上下文规模	意义
2023	GPT-4 8K → 32K	只够塞十几页文档
2024	GPT-4o 128K、Gemini 1.5 Pro 1M	一本 500 页手册能塞进去了
2025~2026	主流旗舰 200K~1M 起步	"整个知识库塞进去"在小规模上确实可行

所以这不是伪问题：在**"文档少、无权限要求、不差钱"**的场景，长上下文就是更简单的方案。RAG 阵营必须回答下面四张牌。

2. 第一张牌：成本——线性计费 vs 常数检索

长上下文每次提问都要**重复支付全部知识**的 token 费用；RAG 每次只检索出相关的几段。

对比项	长上下文直塞	RAG
20 万字知识库，每次问 1 个问题	≈ 每次输入 20 万字（约 13 万 token 级）	≈ 每次输入检索片段 2 千字（约 1.5K token 级）
每问 1000 次的输入成本比	100 倍量级	1
延迟	首 token 延迟随上下文显著增长	检索几十毫秒 + 短上下文生成

(token 数按中文 1 字 ≈ 0.6~0.7 token 粗估，仅示意量级。)

优先级要点： - **P0**：查询频次越高，长上下文的重复成本越致命——企业知识库恰恰是高频场景； - **P1**：成本不仅在线上，也在开发者调试时（每次试验都全量重付）。

3. 第二张牌：注意力——塞进去 ≠ 记得住

Liu et al. 2023 《Lost in the Middle: How Language Models Use Long Contexts》用受控实验证明：关键信息放在上下文**开头或结尾**时表现最好，放在**中间**时性能显著下降（多文档 QA 准确率可跌 20 个百分点以上）。

- 【一句话人话】长上下文不是硬盘，是"注意力带宽有限的阅读理解"——把答案埋在 50 页材料中间，模型真的会漏看。
- 【生活类比】让你开卷考一本 500 页的书，翻到答案页不难；但让你"把整本书默读一遍再闭卷答题"，中间章节必然记不清。

4. 第三张牌：新鲜度——上下文是快照，知识库是活水

- 长上下文：知识变更 = 修改系统提示词/重新分发文档 = 发布动作，且**每一份在用的 prompt 都要同步**；
- RAG：知识更新 = 更新索引数据 = **数据操作**，即时生效、可增量、可回滚。

CTRM 例：2026-09 公司把铜点价的最低点价量从 500 吨改成 300 吨。RAG 方案只需重嵌该规则文档的对应分块；长上下文方案要找到所有塞了旧规则的应用配置逐一修改。

5. 第四张牌：权限——长上下文无法做行级隔离（企业刚需）

这是**最致命**的一张牌：

- RAG：检索前过滤（`where dept = '期货部' AND tenant = 'A公司'`），无权限的 chunk **根本不进入**上下文；
- 长上下文：谁拿到 prompt，谁就看到全部。要么为每个权限组维护一份 prompt（爆炸），要么接受越权泄露。

CTRM 是多租户系统：A 租户的采购合同条款绝不能出现在 B 租户的问答里。结论：**只要知识带权限，长上下文方案在企业场景直接出局**。详见 06 生产篇“检索前 vs 检索后过滤”。

6. 裁定

维度	胜者	说明
单篇/小知识、个人使用	长上下文	简单直接，无需基建
知识量 10 万字级以上	RAG	成本曲线决定
高频查询	RAG	重复计费决定
知识频繁更新	RAG	数据操作 vs 发布动作
多租户/权限隔离	RAG（唯一解）	检索前过滤
单文档深度逐行分析	长上下文	检索会切碎论证链

结论：长上下文是 RAG 的补充，不是替代。 04 进阶篇会给出完整的三方案（RAG vs 长上下文 vs 微调）决策表。

【本节自测】

- 长上下文方案的成本问题本质是什么？** 要点：每次查询重复支付全部知识的输入 token，成本与查询次数相乘；RAG 是常数级检索。
- Lost in the Middle 的结论是什么？** 要点：关键信息在上下文中部时模型利用率显著下降（Liu et al., 2023）；塞得长 ≠ 记得住。

3. **为什么说权限是长上下文的"致命牌"?** 要点: 长上下文无法做行级过滤, 任何塞进 prompt 的内容对读者全可见; 多租户场景直接不可用。
4. **知识规则更新时, 两种方案各需要做什么?** 要点: 长上下文=发布级修改并同步所有 prompt; RAG=数据级增量更新索引, 即时生效。
5. **举一个长上下文反而更合适的场景。** 要点: 单篇长文档的逐行审阅/整体摘要, 无权限要求、低频查询。
6. **给领导的一句话回应"有百万上下文还要 RAG 吗"?** 要点: 成本×频次、注意力稀释、知识新鲜度、权限隔离——四个维度只有 RAG 同时满足, 长上下文只在小而私的低频场景胜出。

02-概念篇：核心概念精讲（12 张概念卡）

【本节问题】① RAG 的最小闭环是哪三步？② 分块、嵌入、向量库分别解决什么？③ 混合检索和重排为什么是"标配组合"？④ 引用接地、上下文打包、新鲜度、幻觉残留这四个质量概念怎么理解？

使用说明：每张卡 = 【一句话人话】+ 【生活类比】+ 要点 + CTRM 映射 + 面试一句话。概念全景见下图。



卡 01：RAG 与三步闭环（索引 / 检索 / 生成）

- 【一句话人话】回答问题前先去资料库里把相关段落翻出来，让模型照着资料答，而不是凭记忆瞎猜。
- 【生活类比】开卷考试：平时不用把书背进脑子（不微调），考试时按题目翻书（检索），照着书页作答（生成），还能注明"答案在第 X 页"（引用）。
- 三步：**索引**（离线：文档→分块→向量→入库）→ **检索**（在线：问题→召回相关块）→ **生成**（块+问题→LLM→带引用答案）。
- CTRM 映射：把《点价操作手册》建成索引，交易员问"点价流程"时系统翻出 3.2 节再作答。
- 面试一句话：RAG 是用工程手段把"知识记忆"从参数转移到外部存储，换取可更新、可授权、可引用。

卡 02：分块 Chunking（固定 / 递归 / 语义 / 父子块）

- 【一句话人话】把长文档切成大小合适的段落，切太碎丢上下文，切太大塞不下也搜不准。

- 【生活类比】切蛋糕：块太大一块吃撑还挑不出想吃的口味；块太小只尝到一粒糖不知道是什么蛋糕。好的切法是沿奶油花纹切（语义边界）。
- 四种策略：

策略	做法	适用	坑
固定长度	每 N 字符切一刀 + 重叠	快糙猛	切断句子/表格
递归	按 <code>\n\n</code> → <code>\n</code> → <code>。</code> 逐级降级切	默认首选	参数需调
语义	相邻句 embedding 相似度骤降处切	长文质量高	索引成本高
父子块	小块用于检索、大块用于生成	手册/条款类	索引结构复杂

- CTRM 映射：合同模板按条款边界切（递归分隔符自定义"第X条"），绝不能把"违约责任"条款切成两半。
- 面试一句话：chunk_size 与 overlap 是 RAG 第一杠杆，没有之一——检索差先查切块。

卡 03：嵌入 Embedding

- 【一句话人话】把一段文字变成一串数字（向量），意思越近的段落数字越接近。
- 【生活类比】给每道菜写一张"口味指纹卡"（麻辣度、甜度、咸度……），以后问"想吃微辣偏甜"时按指纹比对，不用读菜名。
- 要点：bi-encoder 把 query 和文档各自独立编码成向量，速度快适合召回；模型决定语义天花板；**换 embedding 模型必须全量重建索引**。
- CTRM 映射："升贴水怎么算"能命中写着"溢价/折价的确定方式"的块——同义不同词也搜得到，这是 BM25 做不到的。
- 面试一句话：embedding 质量决定召回上限，rerank 只能在上限内排序。

卡 04：向量库

- 【一句话人话】专门存向量和做"最近邻搜索"的数据库，给你"最相似的 k 条"。
- 【生活类比】图书馆的智能检索台：你哼一段旋律，它把最像的几首歌抽出来给你——不用一柜一柜翻。
- 要点：核心是 ANN 近似最近邻算法（HNSW/IVF 等），牺牲一点精度换百倍速度；元数据过滤（租户/部门/时间）是生产必备能力。
- CTRM 映射：选型见 07 对比篇——Java+MySQL 团队最快路径是 pgvector，不必一上来上 Milvus。
- 面试一句话：向量库的本质 = 向量索引 + 元数据过滤 + CRUD，别只比"谁快"。

卡 05：相似度度量（余弦 / 内积 / L2）

- 【一句话人话】判断两个向量"像不像"的三种尺子：夹角、投影重合度、直线距离。

- 【生活类比】两个人口味像不像：看口味指纹的方向像（余弦）、重叠面积（内积）还是离得近（L2）。
- 要点：文本检索事实标准是**余弦相似度**；归一化后的向量余弦与内积等价（多数库归一化后用内积提速）；L2 受向量长度干扰，文本场景少用。
- CTRM 映射：无需选——用库默认余弦/内积即可，知道等价条件即可面试防坑。
- 面试一句话：归一化后 $\cosine \equiv$ 内积；不同库的 metric 参数名不同但语义一致。

卡 06：混合检索（BM25 + 向量 + RRF 融合）

- 【一句话人话】关键词检索和语义检索各搜一遍，把两份结果按排名合到一起。
- 【生活类比】招聘筛简历：HR 按硬关键词筛（"必须有 5 年 CTRM 经验"），主管按气质筛（"像是有大局观的人"），最后老板按两份名单的综合名次定面试名单。
- 要点：BM25 精确匹配强（编号/品名/数值），向量语义泛化强（同义改写）；**RRF (Reciprocal Rank Fusion)** 按名次融合： $score = \sum 1/(k + rank_i)$ ，k 常取 60，无需调分数纲。
- CTRM 映射："CU2509 合约点价保证金比例"——合约代码靠 BM25，"保证金比例"语义靠向量，单路都残。
- 面试一句话：纯向量检索对术语精确匹配是硬伤，混合检索几乎是生产默认。

卡 07：重排 Rerank

- 【一句话人话】先用便宜的召回抓 50 条，再用贵但准的模型给这 50 条重新排座次，取前几名。
- 【生活类比】海选看报名表 30 秒定去留（bi-encoder 快糙），决赛评委逐个上台细看（cross-encoder 慢准）。
- 要点：cross-encoder 把 query 和文档拼在一起过模型，精度高但不能预先索引，只能对候选实时打分；两阶段 = 效率与精度的标准折中。
- 常用模型：bge-reranker-v2-m3（开源、可私有化，常配 bge-m3）；Cohere Rerank（商用 API）。（来源：appscale.blog 2026 embedding 对比，检索于 2026-09）
- CTRM 映射：召回 50 条业务规则块，重排后取 top5 喂给 LLM——答案质量立竿见影。
- 面试一句话：rerank 是 RAG 里"性价比最高的最后一公里"，只跑 top50 控制成本（见 06 生产篇）。

卡 08：Query 改写（HyDE / 多查询 / 指代消解）

- 【一句话人话】用户问得糙，先让 LLM 把问题"翻译"成更容易被检索的形态再去搜。
- 【生活类比】听不懂方言的客服，先让接线员复述成标准普通话再派单。
- 三种：

技术	做法	治什么病
HyDE	让 LLM 先生成一个"假设答案"，拿它去搜	问句与答案文体不匹配

技术	做法	治什么病
多查询 Multi-Query	一个问题改写成 3~5 个角度分别搜，并集召回	表述单一漏召回
指代消解	"它的保证金呢？"→"铜期货的保证金比例是多少？"	多轮对话丢失主语

- CTRM 映射：多轮追问"那升贴水呢？"必须先改写成"铜现货升贴水怎么计算"再检索。
- 面试一句话：query 改写在查询侧投入最小、收益最稳，先做它再谈框架。

卡 09：引用接地 (Grounded Citation)

- 【一句话人话】答案里的每个结论都要标注"来自哪块资料"，模型只能用检索到的内容作答。
- 【生活类比】论文脚注：不是"我听说"，而是"据《点价操作手册》3.2 节"——查证成本交给读者，可信度交给出处。
- 要点：prompt 模板强约束 ("仅依据资料回答，标注 [资料 n]，资料不足则回答不知道")；引用粒度到 chunk + 文档名/章节号。
- CTRM 映射：贸易合规场景答案必须可审计——"该说法出自《2026 点价规则》3.2 节"是上线门槛。
- 面试一句话：接地 = 约束生成范围 + 强制引用 + 允许拒答，三件套缺一不可。

卡 10：上下文打包 (Context Packing)

- 【一句话人话】把重排后的资料块按预算装进 prompt：去重、按相关度排、重要内容放两头。
- 【生活类比】行李箱装箱：限重 20kg (token 预算)，重的垫底 (不重要放中间)，常用放外袋 (关键信息放开头结尾——对抗 Lost in the Middle)。
- 要点：token 预算 = 上下文窗口 - 系统提示 - 预留生成空间；相邻块合并去重；文档名+章节标题作为每块前缀。
- CTRM 映射：升贴水规则表格块 + 计算示例块相邻，打包时合并避免表格被拆散。
- 面试一句话：context packing 是"检索质量"到"生成质量"的搬运环节，搬运损耗常被忽视。

卡 11：知识新鲜度 (Freshness)

- 【一句话人话】RAG 的知识 = 索引里的数据，数据一改答案立刻变新，不用等模型更新。
- 【生活类比】外卖菜单是屏幕上的 (数据)，店里菜谱改了菜单刷新就行；而"背下菜单"的服务员 (模型参数) 要重新培训才会知道新菜。
- 要点：索引更新策略 (增量/全量/淘汰) 见 06 生产篇；**换 embedding 模型是唯一必须全量重建的场景。**
- CTRM 映射：点价规则 9 月 1 日改版，8 月 31 日晚重嵌对应文档，9 月 1 日问答已是新版。
- 面试一句话：新鲜度是 RAG 区别于微调的第一卖点——数据操作 vs 模型训练。

卡 12: 幻觉残留 (Residual Hallucination)

- 【一句话人话】RAG 能大幅减少幻觉，但消灭不了——资料里没有的，模型仍可能“顺势编”。
- 【生活类比】开卷考也得防考生自由发挥：书里只有“升贴水由双方协定”，他偏写出“标准 50 元/吨”。
- 要点：三大残留源——①检索没召回（答非资料）→ 加 rerank/混合检索；②资料冲突（新旧版本并存）→ 索引只留最新版；③模型越界推理 → 引用接地 + 拒答模板 + 低 temperature。
- CTRM 映射：保证金比例写错一位小数就是资金风险——数字类答案必须逐字对照引用块。
- 面试一句话：被问“RAG 还有幻觉吗”——有，先归因到检索/冲突/越界三层再分别给解法。

【本节自测】

1. **RAG 三步闭环是什么？哪步在线哪步离线？** 要点：索引（离线）/检索（在线）/生成（在线）。
2. **递归分块为什么是默认首选？** 要点：兼顾语义边界与实现简单；固定分块易切断语义，语义分块索引成本高。
3. **为什么换了 embedding 模型必须全量重建索引？** 要点：新旧模型的向量空间完全不同，不可比、不可混存。
4. **RRF 的公式和好处？** 要点： $score = \sum 1/(k+rank)$, $k \approx 60$ ；只依赖名次不依赖分数，免去两路分数量纲对齐。
5. **cross-encoder 为什么不能像 embedding 一样预先算好？** 要点：它输入是 query+doc 拼接对，query 到了才能算，无法离线索引。
6. **HyDE 治什么病？** 要点：问句与答案文体不匹配——用假设答案的“文体”去搜答案。
7. **引用接地要同时做哪三件事？** 要点：约束生成范围、强制标注出处、允许答“不知道”。
8. **RAG 幻觉残留的三个来源与对应解法？** 要点：召回缺失→混合检索+rerank；资料冲突→版本治理；模型越界→接地模板+拒答。

03-原理篇：检索管道解剖

【本节问题】① 一份 PDF 是怎么变成向量库里可检索的块的？② 用户一个问题进来，系统内部经过哪些环节？③ 每个环节会以什么方式失败？④ 哪些旋钮值得调、按什么顺序调？

RAG = 两条管道：**索引管道**（离线、一次构建增量维护）与**查询管道**（在线、每次问答跑一遍）。全景见下图，本章逐环节解剖。



一、索引管道：加载 → 清洗 → 分块 → 向量化 → 入库

1.1 加载（Load）

- **职责：**从 PDF/docx/HTML/Confluence/数据库把原始字节变成可解析文本。CTRM 场景四大来源：合同模板（docx/PDF）、业务规则（Word+表格）、操作手册（PDF+截图）、历史点价案例（数据库导出）。
- **工程要点：**PDF 用解析库时保留页码（引用要落到"第几页"）；数据库导出转 Markdown 表格。
- **失败模式：**扫描版 PDF 无文字层（要 OCR）；表格被解析成散乱文字（升贴水规则表全毁）。

1.2 清洗（Clean）

- **职责：**去页眉页脚水印、修复断行、统一全半角、表格转 Markdown、剔除目录与附录噪声。

- **工程要点：**清洗规则**按文档类型配置**（合同和手册的页眉模式不同）；保留标题层级作为后续分块的边界信号和元数据。
- **失败模式：**页眉"第 3 页 共 45 页"混进正文成为"幽灵句子"，被反复召回污染 top-k——这是新手最常踩也最不易察觉的坑。

1.3 分块 (Chunk) —— 第一杠杆

- **职责：**把清洗后的长文切成检索友好的块。
- **策略对比**见下图（详见 02 概念篇卡 02）：



- **参数起点**（中文文本）：chunk_size 400~600 字，overlap 50~100 字，分隔符 ["\n\n", "\n", "。", "，", "；"] 递归切。
- **CTRM 要点：**合同按"第X条"自定义分隔符；带表格的块整体保留不拆。
- **失败模式：**切块切断语义（条款被腰斩）；块太小只有标题没有内容，召回一堆"第 3.2 条 升贴水"却无正文。

1.4 向量化 (Embed)

- **职责：**每块 → embedding 向量（模型选型见 07 对比篇）。
- **工程要点：**批量调用（batch 64~256）；**入库时记录模型名与版本**（换模型=全量重建）；超长块先截断告警而不是静默丢弃。
- **失败模式：**embedding 模型静默截断超长输入（第 1000 字之后的内容根本没进向量，召回后看起来"答非所问"）；索引时用了模型 A、查询时用了模型 B（向量空间不通，全库检索失灵）。

1.5 入库 (Index)

- **职责**：向量 + 原文 + 元数据（文档名/章节/页码/版本/租户/部门/生效日期）写入向量库；同时建 BM25 倒排索引。
- **工程要点**：元数据中 `tenant_id / dept` 字段是权限过滤的生命线（06 生产篇）；原文必须存（生成靠原文，不靠向量反解）。
- **失败模式**：元数据缺失导致权限过滤失效（多租户事故级）；只存向量不存原文。

索引管道失败模式速查表

环节	典型失败	症状	解法
加载	扫描件无文字层	该文档永远召回不到	OCR 预检
清洗	页眉页脚污染	召回块里有"第X页"	按类型配清洗规则
分块	语义被切断	答案缺半句	递归分隔符+overlap
向量化	超长静默截断	块后半内容失效	截断告警
入库	元数据缺权限字段	越权召回	入库 schema 校验

二、查询管道：改写 → 召回 → 融合 → 重排 → 打包 → 生成

2.1 改写 (Rewrite)

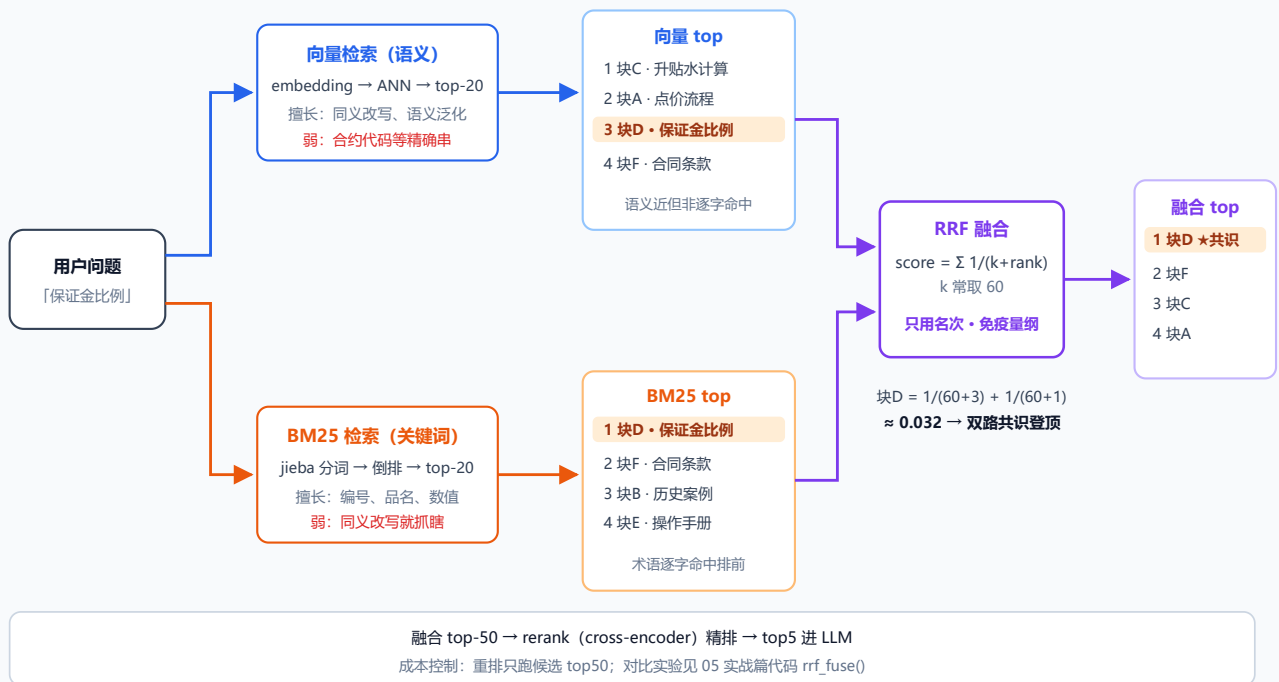
多轮指代消解 + 可选多查询/HyDE（见 02 概念篇卡 08）。**失败模式**：改写丢信息（把"含税价"改没了）；多查询扩大召回也扩大噪声。

2.2 召回 (Recall) —— 双路并行

- **向量路**：改写后 query → embedding → 向量库 top-k（k 取 20~50，给后面重排留余量）。
- **关键词路**：BM25（中文需分词，jieba）top-k 同量级。

混合检索与 RRF 融合原理

BM25 管精确匹配，向量管语义泛化；按名次融合，重排收口



- **失败模式**: 只走向量路, "CU2509"这类合约代码匹配不到 (embedding 对短代码不敏感); query 向量空间与库不一致 (1.4 的坑在查询端爆发)。

2.3 融合 (Fuse)

RRF 按名次合并两路结果: $score(d) = \sum 1/(60 + rank_i(d))$ 。**失败模式**: 用原始分数直接加权 (BM25 分数无上界、余弦在 $[-1,1]$, 量纲不通, 直接相加等于噪声融合)。

2.4 重排 (Rerank) —— 性价比之王

cross-encoder 给融合后的 top50 重新打分, 取 top3~8。**失败模式**: rerank 模型与语料领域差异过大 (金融/贸易术语表现差, 需领域微调或换模型); 对 top500 全量重排 (延迟和费用爆炸, 见 06 生产篇"只跑 top50")。

2.5 打包 (Pack)

去重、相邻合并、按预算装填、关键内容放开头结尾 (对抗 Lost in the Middle)、每块前缀"【文档名·章节·页码】"。**失败模式**: 超预算静默丢块且不告警; 丢的恰好是含答案的块。

2.6 生成 (Generate)

引用接地模板 + 低 temperature + 允许拒答。**失败模式**: 模型无视资料自由发挥 (幻觉残留, 02 概念篇卡 12); 引用编号对不上 (模板没把资料编号传稳)。

查询管道失败模式速查表

环节	典型失败	症状	解法
改写	指代丢失主语	追问时答错对象	多轮消解
召回	术语/代码漏召回	问 CU2509 搜不到	补 BM25 路
融合	分数量纲混加	结果顺序怪异	用 RRF
重排	领域不匹配	好块被压后	换/调 rerank 模型
打包	超预算丢答案块	明明召回了却答错	预算监控+告警
生成	越界发挥	编造数字	接地模板+拒答

三、调优旋钮与顺序

优先级	旋钮	建议动作
P0	chunk_size/overlap/分隔符	检索差先查切块 (02 概念篇卡 02)
P0	清洗规则	有"幽灵句"先去页眉
P1	混合检索 + RRF	加 BM25 路, 治代码/编号漏召回
P1	rerank	接入即涨点, 只跑 top50
P2	query 改写	多轮对话先做指代消解
P2	top_k / 预算	k 给重排留余量; 预算宁大勿丢

调参心法：先用评估集量化 (06 生产篇 RAGAS) , 改一个旋钮测一次——一次改三个等于白调。

【本节自测】

1. **索引管道五步分别是什么？哪一步是"第一杠杆"？** 要点：加载→清洗→分块→向量化→入库；分块。
2. **"幽灵句子"是什么环节的什么失败？** 要点：清洗环节，页眉页脚混入正文被反复召回污染 top-k。
3. **embedding 环节的两个静默坑？** 要点：超长输入静默截断；索引与查询用了不同模型（向量空间不一致）。
4. **RRF 与"分数直接加权"的区别？** 要点：RRF 只用名次，免疫两路分数量纲差异。
5. **top_k 为什么取 20~50 而不是直接取 3？** 要点：给融合和重排留余量，召回阶段宁可多不能漏（召回是上限，重排做精度）。
6. **查询管道哪一步"性价比最高"？** 要点：rerank——接入即涨点，成本可控（只跑 top50）。
7. **为什么原文必须入库，不能只存向量？** 要点：生成靠原文；向量无法可靠反解文本。

04-进阶篇：高级检索与 Agentic RAG

【本节问题】① 父子块和 sentence-window 用什么思路提升检索质量？② GraphRAG 解决什么"向量 RAG 治不了"的问题，2026 年进展如何？③ Agentic RAG 和 Naive RAG 的本质区别是什么、和 003 的工具循环怎么对接？④ 长上下文 vs RAG 的最终裁定怎么给？

一、Advanced RAG：不动模型，动检索结构

1.1 父子块检索 (Parent-Child / Small-to-Big)

- **思路：用小块检索，用大块生成。**子块（如 200 字）向量化，命中后返回其父块（如 1200 字）给 LLM。
- **【生活类比】**用书签找章节：书签很小好翻（子块好检索），翻到后你读的是整章（父块好理解）。
- **收益：**小块的 embedding 更聚焦、相似度更准；大块给 LLM 足够上下文。一举解决"块小搜得准但答不全、块大答得全但搜不准"的两难。
- **CTRM 映射：**《点价操作手册》按节存父块、按段存子块——问"点价流程"命中"3.2 触发点价"小段，返回整节让 LLM 看到完整流程图描述。

1.2 Sentence-Window 检索

- **思路：**按句子向量化；命中某句后，把它的前后 N 句窗口一起返回。本质是"自适应父块"：上下文范围由命中位置动态决定。
- 与父子块的取舍：父子块结构固定、实现简单；sentence-window 粒度更细、上下文更贴身，但窗口大小要调。

1.3 多路召回 (Multi-Route Recall)

- **思路：**同一问题并行走多条检索通道再融合：
- 多查询改写：一个问题 → 3~5 个表述，各自向量检索取并集；
- 多粒度：段落块 + 摘要块 + 表格块分通道；
- 多库：向量库 + BM25 + （业务数据库 SQL 查询，见 1.4 路由）。
- **失败模式：**路数越多融合越难——统一用 RRF；召回噪声随路数上升，重排变成必选项。

1.4 检索路由 (Routing) —— 常被忽视的第一道闸

不是所有问题都该走向量库：

问题类型	路由到	CTRM 例
事实型/规则型	知识库 RAG	"升贴水怎么算"
实时数值	业务 API/SQL	"CU2509 现在的点价是多少"（查行情接口，不查文档！）
操作型	操作手册 RAG	"怎么新增点价单"

问题类型	路由到	CTRM 例
统计型	SQL/报表	"上月点价笔数"

失败模式：把"今天铜价多少"路由到知识库，检索出三天前的行情说明——灾难性回答。路由 = 一个小分类 prompt 或分类模型。

二、GraphRAG：为"关系型问题"补一条路

2.1 它治什么病

向量 RAG 回答的是"某文档里说了什么"（点对点）。但有一类问题是**跨文档关系推理**：

- "哪些客户和同一家供应商同时签了背靠背合同？"
- "A 品种的价格调整条款和 B 品种的有什么关联影响？"

这类问题的答案**不在任何单个块里**，块与块之间需要"连线"——向量检索天然不会连线。

2.2 原理三步

1. **实体抽取：**LLM 从全部文档抽实体（客户/品种/合约/部门）与关系；
2. **社区检测：**图算法（Leiden 等）把紧密关联的实体聚成社区；
3. **社区摘要：**LLM 为每层社区生成摘要；查询时先查社区摘要（全局性问题）或实体邻域（局部性问题），再生成。



2.3 2026-09 现状 (已核实)

事实	来源
原始 GraphRAG (微软 2024) 索引成本极高 (大数据集约 \$33K 量级), 制约普及	Microsoft Research Blog; nemorize.com 2026 RAG 路线图 (检索于 2026-09)
LazyGraphRAG (2025-06 发布) 把 LLM 摘要推迟到查询时: 索引成本降至全量版约 0.1%, 全局查询质量持平、查询成本低约 700 倍	Microsoft Research Blog
微软 GraphRAG 库 2026-03 发布性能优化版本; 相关基准工作 (GraphRAG Benchmark、LinearRAG) 被 ICLR 2026 接收; MIT 开源	microsoft.github.io/graphrag
混合路由成为行业共识 : 简单事实→向量 RAG; 多跳关系→GraphRAG; LazyGraphRAG 让全局查询路径可负担	bestaiweb.ai / nemorize.com (检索于 2026-09)
轻量路线 LightRAG (HKUDS) 走"轻索引+双层检索", 2026-04 发布 v1.4.15 (EMNLP 2025 论文)	LightRAG GitHub

- **采用建议**: 默认不上 GraphRAG。先问自己"问题里有没有跨文档多跳关系"; 有, 且 LazyGraphRAG 级成本可接受, 再上。CTRM 知识库第一版用不上——合同实体图谱是第二阶段的事。

三、Agentic RAG: 检索从"管线"降级为"工具"

3.1 本质区别

	Naive/Advanced RAG	Agentic RAG
控制流	固定管线, 代码写死	agent 的 ReAct 循环决定
检索时机	每问必查	模型决定何时查 (常识题不查)
检索次数	一次	可多轮 (查→读→发现缺→再查)
工具面	只有向量库	向量库 + SQL + API + Web 多工具选择

3.2 与 003 的衔接

Agentic RAG 没有任何新协议——它就是 003 专题的**工具循环 (ReAct)** 套用到"检索"这个工具上:

Thought: 用户问"上月铜点价均价与升贴水关系"—需先查规则定义升贴水, 再查 SQL 拿上月数据
Action: search_knowledge_base("升贴水的定义与计算")
Observation: 升贴水 = 现货价 - 合约价 ... (引用块)
Thought: 规则清楚了, 还缺上月点价数据
Action: query_ctrm_db("SELECT avg(price) FROM dianjia WHERE month='2026-08' AND variety='CU'")
Observation: 均价 75230 元/吨
Thought: 两类信息齐了 → 生成带引用的最终答案

关键工程点：**每个工具要有清晰的 description**（003 工具规范），否则 agent 会把"查行情"路由到知识库；多轮检索要有轮数上限与成本预算。

3.3 什么时候值得 Agentic 化

- P0 信号：问题需要**组合多源**（文档+数据库+API）；
- P1 信号：单次检索命中率低，需要"查了再查"；
- 反指标：问题单一固定、延迟敏感——固定管线更快更便宜，别为 Agentic 而 Agentic。

四、长上下文 vs RAG：最终裁定

三张决定性牌（成本/新鲜度/权限）+ 一个实用结论（01 问题篇详证）：

决策因素	倾向 RAG	倾向长上下文
知识量	大（10 万字级+）	小（几篇文档）
查询频次	高频	低频
知识更新	频繁	几乎不变
权限隔离	需要（唯一解）	无要求
引用审计	需要	无要求
任务形态	查找型问答	单篇深度通读

裁定：三者只要占一条就回不到"纯长上下文"；权限是硬门槛。 2026 年的正确姿势是混合：小而稳的常用资料直接常驻 system prompt（省检索），大而活的知识库走 RAG，实时数值走 API——由路由层统一调度。

【本节自测】

1. **父子块检索解决什么两难？** 要点：块小搜得准但答不全、块大反之；小块检索、大块生成，两全。
2. **sentence-window 与父子块的区别？** 要点：按句索引、命中后动态取前后 N 句窗口；粒度更细但窗口参数需调。
3. **为什么"今天铜价多少"不能路由到知识库？** 要点：实时数值在业务系统不在文档；路由到 RAG 只能答出过期数据。
4. **GraphRAG 适合什么问题？举例一个 CTRM 关系型问题。** 要点：跨文档多跳关系推理；如"哪些客户与同一供应商同时存在背靠背合同"。
5. **LazyGraphRAG 的核心改动与效果？** 要点：LLM 摘要从索引期推迟到查询期；索引成本降至全量版约 0.1%，全局查询质量持平（来源：Microsoft Research Blog）。
6. **Agentic RAG 与 003 专题的关系？** 要点：无新协议，就是 ReAct 工具循环 + 把检索作为一个工具；重点是工具 description 写清与多轮成本控制。
7. **"RAG 不会死"的三张牌是什么？** 要点：成本（线性重复计费）、新鲜度（数据操作 vs 发布动作）、权限（检索前过滤是唯一解）。

05-实战篇：从零搭建贸易知识库问答（Python 全代码）

【本节问题】① 不用 LangChain 框架，一个最小可跑的 RAG 需要多少代码？② 递归分块、BM25、RRF 手写出来长什么样？③ rerank 怎么接（商用 API 和本地模型两条路）？④ CTRM 多租户权限过滤在代码里怎么落？

0. 目标与架构

跑通后的效果：

问：点价流程是什么？
答：点价流程分四步：①确认点价指令.....[来源：点价操作手册 · 3.2节]
问：升贴水怎么算？
答：升贴水 = 现货成交价 - 对应合约结算价，买卖双方按品种协定.....[来源：业务规则汇编 · 2.1节]

端到端架构（本章代码逐块实现下图每一格）：



1. 环境准备

```
# Python 3.10+
pip install openai chromadb rank_bm25 jieba
# 可选 rerank 二选一:
pip install cohere # 路 A: Cohere Rerank API
pip install sentence-transformers # 路 B: 本地 bge-reranker-v2-m3
```

2. 全量代码（单文件 ctrm_rag.py，约 260 行）

```
# -*- coding: utf-8 -*-
"""
CTRM 贸易知识库问答：最小可跑 RAG
- 向量库：chromadb          - 关键词：rank_bm25 + jieba
- 融合：  手写 RRF          - 重排：cohere 或本地 bge-reranker（可选）
- embedding：任意 OpenAI 兼容接口（OpenAI / 通义 / 智谱 / DeepSeek 等均可，
  修改 base_url 与 model 即可；国内私有化推荐 bge-m3，见 07 对比篇）
"""

import os
import jieba
import chromadb
from openai import OpenAI
from rank_bm25 import BM25Okapi

# ===== 0. 配置 =====
EMBED_MODEL = os.getenv("EMBED_MODEL", "text-embedding-3-small")
CHAT_MODEL  = os.getenv("CHAT_MODEL", "gpt-4o-mini")  # 换成你的兼容端点模型
TOP_K_RECALL = 20  # 每路召回条数（给融合/重排留余量，见 03 原理篇）
RRF_K        = 60  # RRF 平滑常数
RERANK_N     = 5   # 最终喂给 LLM 的条数
CONTEXT_BUDGET = 3000 # 打包 token 预算（按字符粗估，中文 1 字≈0.7 token）

llm = OpenAI()      # 需要 OPENAI_API_KEY；兼容端点再传 base_url=...
emb = OpenAI()      # 可指向不同 base_url

# ===== 1. 语料：CTRM 示例文档（生产中来自 docx/PDF/DB 导出） =====
DOCS = [
    {"id": "d1", "title": "点价操作手册", "sec": "3.2 触发点价", "dept": "trading",
     "text": "点价流程分四步：①买方在合约到期前向交易部提交点价指令，注明品种、合约月份与点价数量；"
            "②交易员在期货市场按指令执行买入/卖出保值头寸；③成交后系统自动生成分批点价确认单，"
            "④双方经办人当日复核签字；⑤财务按确认单登记应收应付并释放保证金占用。单笔点价数量不得低于 300 吨。"},
    {"id": "d2", "title": "业务规则汇编", "sec": "2.1 升贴水", "dept": "trading",
     "text": "升贴水的确定：升贴水 = 现货成交价 - 对应期货合约结算价。正值为升水，负值为贴水。"
            "买卖双方应在合同订立时按品种协定基准升贴水；实际结算升贴水按交割地与基准地运费差异调整。"},
    {"id": "d3", "title": "业务规则汇编", "sec": "2.4 保证金", "dept": "risk",
     "text": "点价业务保证金比例：一般客户按合约价值的 10% 收取；信用评级 AA 及以上客户可下浮至 8%；"
            "连续两个交易日涨停的品种次日上调 3 个百分点。保证金低于维持线时系统自动推送追保通知。"},
    {"id": "d4", "title": "合同模板说明", "sec": "5.1 价格调整条款", "dept": "legal",
     "text": "背靠背合同的价格调整条款：上游采购价与下游销售价须采用同一期货合约月份为定价基准，"
            "调价周期一致，不得出现单边敞口。若上游为含税价而下游为不含税价，应在补充协议中明确税率承担方。"},
    {"id": "d5", "title": "历史点价案例", "sec": "2025Q3-CU-017", "dept": "trading",
     "text": "2025 年三季度 CU2509 合约案例：客户在贴水扩大至 180 元/吨时锁定 1200 吨点价，"
            "较均价提前两周完成，规避了后续贴水收窄风险。经验：贴水扩大期分 3 批点价，每批间隔 2 个交易日。"},
]

# ===== 2. 递归分块（02 概念篇卡 02：块 400 字 + overlap 80，按语义分隔符降级） =====
def recursive_split(text: str, size: int = 400, overlap: int = 80) -> list[str]:
    if len(text) <= size:
        return [text]
    for sep in ["\n\n", "\n", "。", "，", "; "]:
        parts = [p for p in text.split(sep) if p.strip()]
        if len(parts) > 1:
            chunks, cur = [], ""
            for part in parts:
                if len(cur + part) > size:
                    chunks.append(cur)
                    cur = part
                else:
                    cur += part
            chunks.append(cur)
            return chunks
```

```

        for p in parts:
            cand = (cur + sep + p) if cur else p
            if len(cand) > size and cur:
                chunks.append(cur)
                cur = cur[-overlap:] + p      # overlap 保证边界语义连续
            else:
                cur = cand
        if cur:
            chunks.append(cur)
        return [c for c in chunks if c.strip()]
# 无分隔符可用：硬切
step = size - overlap
return [text[i:i + size] for i in range(0, len(text), step)]

# ===== 3. embedding (批量调用) =====
def embed(texts: list[str]) -> list[list[float]]:
    resp = emb.embeddings.create(input=texts, model=EMBED_MODEL)
    return [d.embedding for d in resp.data]

# ===== 4. 索引管道：分块 → 向量化 → 入库（向量库 + BM25） =====
def build_index():
    chunks = []      # 每条: {doc_meta..., "text": 块原文}
    for d in DOCS:
        for i, seg in enumerate(recursive_split(d["text"])):
            chunks.append(**d, "chunk_id": f'{d["id"]}-{i}', "text": seg))

    client = chromadb.Client()
    col = client.get_or_create_collection("ctrm_kb", metadata={"hnsw:space": "cosine"})
    col.upsert(
        ids=[c["chunk_id"] for c in chunks],
        embeddings=embed([c["text"] for c in chunks]),
        documents=[c["text"] for c in chunks],
        metadatas=[{"title": c["title"], "sec": c["sec"], "dept": c["dept"]} for c in chunks],
    )

    tokenized = [list(jieba.lcut(c["text"])) for c in chunks]
    bm25 = BM25Okapi(tokenized)
    return col, bm25, chunks

# ===== 5. 双路召回（检索前权限过滤！） =====
def vector_search(col, query: str, dept: str, k=TOP_K_RECALL):
    qv = embed([query])[0]
    r = col.query(query_embeddings=qv, n_results=k,
                  where={"dept": dept})      # ← 检索前过滤：无权限的块根本不进上下文
    return list(zip(r["documents"][0], r["ids"][0]))

def bm25_search(bm25, chunks, query: str, dept: str, k=TOP_K_RECALL):
    toks = list(jieba.lcut(query))
    scores = bm25.get_scores(toks)
    ranked = sorted(zip(scores, chunks), key=lambda x: -x[0])
    return [(c["text"], c["chunk_id"]) for s, c in ranked[:k] if s > 0 and c["dept"] == dept]

# ===== 6. 手写 RRF 融合（score = Σ 1/(K + rank)） =====
def rrf_fuse(list_a, list_b, k=RRF_K):
    pool = {}
    for rank, (_text, cid) in enumerate(list_a, 1):

```

```

        pool[cid] = pool.get(cid, 0.0) + 1.0 / (k + rank)
    for rank, (_text, cid) in enumerate(list_b, 1):
        pool[cid] = pool.get(cid, 0.0) + 1.0 / (k + rank)
    return sorted(pool.items(), key=lambda x: -x[1])          # [(chunk_id, rrf_score)]

# ===== 7. 重排（可选，两选一；不装包则跳过，RRF 名次兜底） =====
def rerank(query, candidates, chunks):
    text_of = {c["chunk_id"]: c["text"] for c in chunks}
    docs = [text_of[cid] for cid, _ in candidates]

    # 路 A: Cohere Rerank API
    if os.getenv("COHERE_API_KEY"):
        import cohere
        co = cohere.Client(os.getenv("COHERE_API_KEY"))
        res = co.rerank(model="rerank-multilingual-v3.0",
                        query=query, documents=docs, top_n=RERANK_N)
        return [docs[r.index] for r in res.results]

    # 路 B: 本地 bge-reranker-v2-m3（首次运行自动下载，需 GPU 或耐心 CPU）
    global _cross_encoder
    try:
        _cross_encoder
    except NameError:
        from sentence_transformers import CrossEncoder
        _cross_encoder = CrossEncoder("BAAI/bge-reranker-v2-m3")
    scores = _cross_encoder.predict([(query, d) for d in docs])
    top = sorted(range(len(docs)), key=lambda i: -scores[i])[:RERANK_N]
    return [docs[i] for i in top]

# ===== 8. 打包（预算内装填，来源前缀显式编号） =====
def pack_context(docs: list[str]) -> str:
    out, used = [], 0
    for i, d in enumerate(docs, 1):
        if used + len(d) > CONTEXT_BUDGET:
            break          # 宁可少放，不可静默丢答案块而不告警
        out.append(f"资料{i}] {d}")
        used += len(d)
    return "\n\n".join(out)

# ===== 9. 生成（引用接地三件套：限范围 + 标来源 + 允许拒答） =====
SYSTEM = (
    "你是中基集团 CTRM 贸易知识库助手。只依据【资料】回答："
    "每个结论标注 [资料n]；资料不足以回答时，回答'知识库中未找到依据'。禁止编造数字。"
)

def answer(query: str, dept: str = "trading"):
    col, bm25, chunks = build_index()          # demo 每次重建；生产中索引常驻，见 06 生产篇
    hits = rrf_fuse(vector_search(col, query, dept),
                    bm25_search(bm25, chunks, query, dept))
    try:
        docs = rerank(query, hits, chunks)     # rerank 可选，失败则用 RRF 前 N 兜底
    except Exception:
        text_of = {c["chunk_id"]: c["text"] for c in chunks}
        docs = [text_of[cid] for cid, _ in hits[:RERANK_N]]

    ctx = pack_context(docs)

```

```

msg = llm.chat.completions.create(
    model=CHAT_MODEL, temperature=0.1,
    messages=[{"role": "system", "content": SYSTEM},
               {"role": "user", "content": f"【资料】\n{ctx}\n\n【问题】{query}"},
    )
return msg.choices[0].message.content

# ===== 10. 演示 =====
if __name__ == "__main__":
    for q in ["点价流程是什么?", "升贴水怎么算? "]:
        print(f"\n问: {q}\n答: {answer(q)}")

```

3. 代码与原理的对照表

代码位置	对应原理 (03 原理篇)	关键点
<code>recursive_split</code>	索引管道·分块	分隔符降级 + overlap
<code>embed</code>	索引管道·向量化	批量调用; 生产要记录模型版本
<code>build_index</code>	索引管道·入库	向量 + 原文 + 元数据三样都存
<code>vector_search</code> 的 <code>where</code>	权限过滤	检索前过滤 , 无权限块不进上下文
<code>bm25_search</code>	查询管道·召回	jieba 分词; 只对同 dept 打分
<code>rrf_fuse</code>	查询管道·融合	只用名次, 免疫量纲
<code>rerank</code>	查询管道·重排	cross-encoder 只跑候选集
<code>pack_context</code>	查询管道·打包	预算装填 + 来源编号
SYSTEM 模板	查询管道·生成	接地三件套

4. 从 demo 到生产的差距清单

#	demo 做法	生产做法	见
1	每次问答重建索引	索引常驻 + 增量更新	06 生产篇
2	<code>dept</code> 硬编码	租户/部门从登录态注入	06 生产篇
3	无评估	RAGAS 四指标 + 评估集	06 生产篇
4	无缓存	query 向量缓存	06 生产篇
5	单文件	服务化 (Java 侧可用 HTTP 包一层给 Vue 2.7 前端)	—

给 Java 团队的落地建议: 把 `ctrm_rag.py` 用 FastAPI 包成 `/ask` 接口, CTRM 的 Vue 2.7 前端与 Java 11 后端照常调用——RAG 是独立服务, 不需要改写主系统。

【本节自测】

1. **代码里权限过滤在哪一行实现？为什么放在召回阶段而不是生成阶段？** 要点：`vector_search` 的 `where={"dept": dept}`；检索前过滤保证无权限内容根本不进入候选池（检索后过滤会造成 top-k 缺口）。
2. `recursive_split` 的 `overlap` 起什么作用？ 要点：相邻块重叠保证被切断的边界语义在两块中都完整。
3. `rrf_fuse` 为什么只用名次不用原始分数？ 要点：BM25 分数无上界、余弦相似度有界，量纲不同不能相加。
4. **rerank 失败时代码怎么兜底？** 要点：`except` 后直接取 RRF 融合前 `RERANK_N` 条——重排是增强不是依赖。
5. `pack_context` 超预算时为什么 `break` 而不是截断？ 要点：截断会产生半块污染上下文；`break` 丢弃整块且生产中应告警。
6. **把 demo 变生产，最先要改的三件事？** 要点：索引常驻+增量更新、权限从登录态注入、建立评估集与指标。

06-生产篇：评估、索引维护与成本

【本节问题】① RAG 上线后怎么量化"答得好不好"? ② 评估集怎么建? ③ 文档天天改, 索引怎么更新? ④ 多租户权限过滤为什么必须检索前? ⑤ token 成本从哪里省?



一、评估：RAGAS 四指标

RAGAS 是目前事实标准的 RAG 自动评估框架。四个指标各管一段管道，低分能直接定位病灶：

指标	测什么	计算思路	低分说明哪坏了
Faithfulness 忠实度	答案是否忠于资料	把答案拆成陈述句，逐条核对能否由召回内容支持	生成端越界 (幻觉) ——查接地模板/temperature
Answer Relevancy 答案相关性	答案是否切题	由答案反向生成若干"可能的问 题"，与原问题算相似度	生成端答偏——查 prompt 约束
Context Precision 上下文精确率	召回块里相关块是否排在前面	召回块按与金标准的贡献排序，算排名加权精度	重排/融合失效——好块被压后
Context Recall 上下文召回率	金标准答案的信息是否都被召回覆盖	金标准答案的句子与召回块做覆盖匹配	检索端漏召回——查分块/混合检索

- 【一句话人话】faithfulness 管"没瞎编"，relevancy 管"答的是所问"，precision 管"给模型的资料干净"，recall 管"该给的资料给了"。

- 【生活类比】阅卷：没抄错书（忠实）、答的是这道题（切题）、草稿纸没杂讯（精确）、需要的参考页都翻到了（召回）。
- **定位口诀**：recall 低修索引（分块/召回），precision 低修重排，faithfulness 低修生成约束，relevancy 低两头都查。

二、评估集建设

没有评估集的调参都是玄学。最小可行评估集：

组成	数量起点	来源
真实问题	100~200	客服记录/业务群提问日志采样（CTRM：业务群里天天有人问规则）
金标准答案	同上	业务专家确认（或 LLM 初稿+人工校对）
金标准块	每题 1~3 个	专家指出"答案应出自哪段"（context recall 必需）
拒答题	占 10~15%	故意问知识库里没有的，验证"不知道"能力
困难题	占 20%	多跳/表格/数值精确匹配，防止评估集太软

要点：评估集**版本化**（同 06 索引变更一起管理）；每次改动管道跑全量回归；指标趋势比单次绝对值重要。

三、索引更新策略

策略	触发条件	做法	风险
增量更新	文档内容修订（最常见）	按 updated_at 找变更文档 → 删旧块 → 重新分块嵌入 upsert	新旧版本并存导致答案冲突
全量重建	换 embedding 模型/换分块策略（唯一必选场景）	新 collection 双写 → 灰度切流 → 下线旧库	成本高；务必先在新库跑评估回归
淘汰	文档下线/过期	按 doc_id 删全部块（含 BM25 侧）	漏删某路索引导致"僵尸块"仍被召回

- 【生活类比】增量=单点改菜谱改那一页；全量=换菜单版式整本重印（字体变了所有页都要重排——embedding 换模型同理，向量空间整个变了）；淘汰=下架的菜要从所有菜单渠道撤掉。
- **版本治理铁律**：同一文档同一时间**只允许一个生效版本**在索引里，旧版必须淘汰——"新旧并存"是答案自相矛盾的头号根因（02 概念篇卡 12 幻觉残留源②）。
- CTRM 例：9 月 1 日点价新规生效，8 月 31 日晚增量更新新文档、淘汰旧版块，并跑一遍"点价流程"评估子集回归。

四、权限过滤：检索前 vs 检索后

	检索前过滤（query 时 where）	检索后过滤（取回再删）
做法	向量库查询带 <code>where={"tenant":..., "dept":...}</code>	先取 top-k 再在代码里删无权限项
top-k 缺口	无——k 条全部有效	有——若 10 条中 7 条无权限，实际只剩 3 条，答案质量骤降
信息泄露	无权限块从未进入候选	无权限块已出现在管道中间（日志/调试时易泄露）
性能	索引支持元数据过滤时开销小	少量预取开销

裁定：多租户（CTRM 就是）必须检索前过滤。 检索后过滤仅可用于权限维度单一、可预取超大 k 的内部场景——而且还要防中间态泄露。配合 03 原理篇“入库 schema 校验”（权限字段缺失的块禁止入库）形成闭环。

五、成本优化

按性价比排序的清单：

优先级	手段	原理	收益量级
P0	query 向量缓存	同 query 不重复 embedding；命中率随用户量上升	embedding 调用省 30%+
P0	rerank 只跑 top50	cross-encoder 最贵，候选集封顶	重排成本与延迟可控
P1	embedding 批处理	batch 64~256 摊薄网络往返	索引吞吐翻倍
P1	生成端缓存	（问题，知识版本）为 key 缓存答案；知识更新时失效	高频 FAQ 场景省生成大头
P2	小模型路由	简单改写/分类用小模型，生成才用大模型	token 费用分级
P2	检索前过滤顺带省 token	权限过滤收窄候选，打包更聚焦	间接收益

成本监控最小集：每请求记录 `召回数/重排候选数/输入 token/输出 token` 四个数字，周报看趋势——成本劣化往往来自索引悄悄膨胀（忘淘汰旧块）。

六、上线检查单（浓缩版）

1. 评估集 ≥ 100 题目含拒答题，四指标有基线；
2. 多租户检索前过滤 + 入库 schema 校验；
3. 文档更新 SOP：增量更新 + 版本淘汰 + 回归评估；
4. rerank 候选封顶 + query 向量缓存；

5. 答案带来源（文档/章节/页码），可审计。

【本节自测】

1. **RAGAS 四指标分别是什么？各对应哪段管道？** 要点：faithfulness（生成/越界）、answer relevancy（生成/切题）、context precision（重排融合）、context recall（检索召回）。
2. **context recall 低，先修什么？** 要点：检索端——分块策略、召回路数、混合检索；不是先改 prompt。
3. **哪种情况必须全量重建索引？** 要点：换 embedding 模型（或换分块策略）；向量空间不可比，且要做灰度与回归。
4. **"新旧版本并存"会导致什么？铁律是什么？** 要点：答案自相矛盾（幻觉残留源之一）；同一文档同时只允许一个生效版本。
5. **检索后过滤的两大问题？** 要点：top-k 缺口（无权限项占坑）与中间态泄露；多租户必须检索前。
6. **为什么 rerank 只跑 top50？** 要点：cross-encoder 逐对打分最贵；50 条已覆盖融合后有效候选，封顶控成本。
7. **评估集为什么必须含拒答题？** 要点：验证"知识库没有时答不知道"的能力——企业场景宁可答不能编。

07-对比篇：方案选型与向量库对比

【本节问题】① 五个主流向量库怎么选？② embedding 模型 2026 年的选型格局是什么？③ rerank 模型配谁？④ "RAG vs 长上下文 vs 微调"这张决策表怎么用？

一、向量库选型

	pgvector	chromadb	milvus	qdrant	elasticsearch
定位	Postgres 扩展，事务型应用内嵌	轻量嵌入式/原型首选	分布式大规模专用库	高性能 Rust 系专用库	全文检索老牌 + 向量补充
优点	与业务数据同库、SQL join、事务、备份复用	几行代码起服务，上手最快	十亿级向量、生态成熟	内存效率高、过滤快	BM25 原生最强，混合检索一体
缺点	十亿级向量吃力	不适合大规模生产	运维组件多 (etcd 等)	社区比 ES/Milvus 小	向量能力是"附加项"，资源占用大
规模甜区	百万~千万级	原型/单机	亿级+	千万~亿级	千万级 + 强关键词需求
运维成本	极低 (已有 PG 则为零)	低	高	中	中高 (多数公司已 有)
Java 团队友好度	★★★★★ (JDBC 直连)	★★★	★★★	★★★★ (REST/gRPC)	★★★★★ (ES 客户端成熟)

给 CTRM (Java 11 + MySQL) 的选型路径：- P0 起步：PostgreSQL + pgvector——与业务同库、权限过滤用熟悉的 SQL where，运维零新增；数据量百万级向量内完全够用。- 若坚持纯 MySQL 栈：MySQL 9 已引入 VECTOR 类型，但生态与工具链尚浅，截至 2026-09 未见大规模生产案例的权威披露，建议观望或直接 用 PG。- 升舱触发条件：向量过亿、QPS 高、需要复杂多租户隔离 → Milvus/Qdrant；关键词检索一直是主场景 → ES 一体化混合检索。

二、embedding 模型选型 (2026-09 已核实)

模型	维度	最大长度	中文	部署	参考价	适用
BAAI/bge-m3	1024	8192	强	开源可私有化 (单卡 8GB 级)	免费 (付 GPU)	中文企业库默认首选；稠密+稀疏+多向量三合一
OpenAI text-embedding-3-large	256~3072	8191	一般	API	\$0.13/1M tok	规模化安全默认，生态最深

模型	维度	最大长度	中文	部署	参考价	适用
OpenAI text-embedding-3-small	512~1536	8191	一般	API	\$0.02/1M tok	性价比 API 之选
Voyage-3 系列	1024~2048	16K~32K	一般	API	~\$0.06/1M tok	检索基准领先，长文档
Cohere embed-v3/v4	1024	512~10240	一般	API/可私有	\$0.10/1M tok	与 Cohere Rerank 端到端
Qwen3-Embedding (阿里)	可变	16384	强	开源	免费	中文/多语言、长文本、代码

来源：pecollective.com 《What Is the Best Embedding Model? A Direct Answer for 2026》、
 appscale.blog 2026 模型对比、openllm.wavise.com 选型指南、CSDN 《主流嵌入模型总览
 (2026) 》，均检索于 2026-09；价格为各来源引用的公开牌价，以厂商官网为准。

选型决策树 (CTRM 语境)：1. 数据能否出境？不能 → 开源私有化：**bge-m3**（或 Qwen3-Embedding）；2. 能出境且要省事 → text-embedding-3-small 起步，质量不够换 3-large；3. 检索质量是瓶颈且预算可加 → 试 voyage-3 系列，与 baseline 跑评估集对比（注意：MTEB 高 ≠ 你的语料上高，**必须用自己的评估集验证**）。

rerank 配对：bge-m3 → bge-reranker-v2-m3（开源配对，两阶段管线边际成本近零）；OpenAI/Cohere API → Cohere Rerank 或本地 bge-reranker（来源：appscale.blog，检索于 2026-09）。

三、"RAG vs 长上下文 vs 微调"决策表

维度	RAG	长上下文直塞	微调
改的是什么	外部知识（数据）	单次输入（prompt）	模型权重
知识更新	即时（索引操作）	随 prompt 重发	重新训练（周级）
成本曲线	建索引一次+查询便宜	每次查询线性付费	训练贵、推理不变
权限隔离	强（检索前过滤）	弱	无
可引用审计	强（引用接地）	弱	无
治幻觉	减少但不消灭	减少但不消灭	改风格/格式最有效
适合	企业知识问答、多租户	单篇深读、个人低频	输出风格/格式/领域语言
不适合	学风格、实时数值	高频+大库+权限	教新知识（易过时且贵）

【一句话人话】微调是"培训员工"（改脑子，贵、慢、教新知识会过时）；RAG 是"给他配资料柜"（改资料，快、便宜、可授权可审计）；长上下文是"把资料全摊桌上"（随用随摊，桌子有限还费钱）。**三者常组合，不**

是三选一：微调定风格 + RAG 供知识 + 长上下文处理单篇深读。

四、CTRM 推荐配置（结论速查）

层	推荐	备选
向量库	PostgreSQL + pgvector	Qdrant（升舱）
embedding	bge-m3 私有化	text-embedding-3-small
rerank	bge-reranker-v2-m3	Cohere Rerank
关键词	BM25（pg 内 tsearch 或独立）	ES 混合
生成 LLM	内部合规允许的任意模型	—
评估	RAGAS + 自建评估集	—

【本节自测】

1. **pgvector 为什么是 Java+MySQL 团队的起步首选？** 要点：与业务同库（SQL join、事务、备份复用）、运维零新增、权限过滤用 SQL；百万级够用。
2. **什么时候必须离开 pgvector 升舱？** 要点：亿级向量/高 QPS/复杂隔离需求 → Milvus/Qdrant。
3. **中文私有化场景的 embedding 默认选择及理由？** 要点：bge-m3——中文强、8192 长度、稠密+稀疏+多向量、开源免费可单卡部署（来源见上表）。
4. **MTEB 分数高的模型可以直接上线吗？** 要点：不行，MTEB ≠ 你的语料；必须用自建评估集对比验证。
5. **bge-m3 该配哪个 rerank？为什么？** 要点：bge-reranker-v2-m3，同源配对、cross-encoder、自托管零边际 API 成本。
6. **"教模型用我们的合同语气起草条款"该用哪个方案？** 要点：微调（风格/格式是权重的事），RAG 治不了语气。
7. **"上月点价笔数统计"该走哪条路？** 要点：都不是——路由到业务 SQL/报表（04 进阶篇路由表）。

08-自测篇：题库与面试题

【本节问题】① 概念题我能不能秒答？② 原理题能不能讲清"为什么"？③ 工程题能不能落到 CTRM 场景？④ 面试大题怎么组织出结构化回答？

使用说明：先遮住要点自测；答错回到对应篇目（题后括号标注）。面试大题在文末，用"总—分—例"结构作答。

一、概念判断题（对/错并说理由）

1. RAG 微调了大模型的参数，所以知识能更新。（）[00 背景篇] 要点：错。RAG 不动参数，知识在外部索引里；"改数据不改模型"正是其卖点。
2. chunk 越大，召回越准。（）[02 概念篇] 要点：错。大块 embedding 语义被稀释，相似度变糊；小块检索大块生成（父子块）才是解。
3. 有了百万 token 上下文，RAG 就过时了。（）[01 问题篇] 要点：错。成本×频次、注意力稀释、新鲜度、权限四关过不去；权限是硬门槛。
4. 余弦相似度和内积在任何情况下等价。（）[02 概念篇] 要点：错。向量归一化后才等价。
5. rerank 模型可以像 embedding 一样预先算好存库。（）[02 概念篇] 要点：错。cross-encoder 输入是 (query, doc) 对，query 到了才能算。

二、原理简答题

6. 画出/口述 RAG 两条管道的全部环节。[03 原理篇] 要点：索引=加载→清洗→分块→向量化→入库；查询=改写→召回（双路）→融合（RRF）→重排→打包→生成；能报出每步一个失败模式加分。
7. BM25 和向量检索各自的强项？为什么要融合？[03 原理篇] 要点：BM25 强在精确匹配（编号/品名/数值），向量强在语义泛化（同义改写）；单路都有盲区，融合互补。
8. RRF 公式及其相对分数加权的优势？[03 原理篇] 要点： $score = \sum 1/(k+rank)$ ， $k \approx 60$ ；只用名次，免疫两路分数量纲差异。
9. 父子块检索为什么能同时提升检索精度和回答质量？[04 进阶篇] 要点：小块 embedding 聚焦→检索准；命中后返回父块→生成上下文足。
10. GraphRAG 解决向量 RAG 的什么盲区？LazyGraphRAG 破了什么局？[04 进阶篇] 要点：跨文档多跳关系推理（答案不在单块内）；LazyGraphRAG 把 LLM 摘要推迟到查询时，索引成本降至全量版约 0.1%（Microsoft Research Blog）。

三、工程实践题

11. 检索回来的块里总出现"第 3 页 共 45 页"，怎么办？[03 原理篇] 要点：清洗环节按文档类型去页眉页脚；这是"幽灵句"污染 top-k。

12. 用户问"升贴水怎么算", 答出来的公式是旧版规则。根因与修复? [06 生产篇] 要点: 新旧版本并存; 铁律=同一文档仅一个生效版本, 增量更新+淘汰旧块。

13. 多租户下 A 租户用户偶尔搜到 B 租户的合同条款, 怎么排查? [06 生产篇] 要点: 查权限过滤层级——入库 schema 校验权限字段→检索前 where 过滤; 绝不能只做检索后过滤 (top-k 缺口+中间态泄露)。

14. RAGAS 评估 faithfulness 低、context recall 高, 修哪? [06 生产篇] 要点: 资料给了但答案越界——修生成端: 接地模板、低 temperature、允许拒答。

15. embedding 调用费用翻倍, 三个最可能的省钱动作? [06 生产篇] 要点: query 向量缓存、批处理、(若换 API) 选低价模型/私有化; rerank 候选封顶管的是重排费用。

四、场景设计题

16. 为 CTRM 设计知识库的分块方案 (合同模板/规则手册/历史案例三类)。[02/03] 要点: 合同按"第X条"自定义分隔符递归切; 规则手册含表格→表整体成块+父子块; 案例按单条案例切并带时间/品种元数据; 统一存元数据 (dept/tenant/版本/生效日)。

17. 业务方要求"答案必须可追溯到条款原文", 怎么落地? [02/05] 要点: 入库时保留章节/页码元数据; 打包时来源前缀编号; 生成模板强制 [资料n] 引用 + 拒答; 前端把引用渲染成可点击跳转。

18. 领导问"直接把知识库塞给模型不是更快吗", 给出三分钟回应。[01] 要点: 成本 (每次查询重复付费×高频)、Lost in the Middle (长≠记得住)、新鲜度 (数据操作 vs 发布动作)、权限 (唯一解) 四点+结论"小而私低频可直塞, 企业库必须 RAG"。

五、面试大题

19. (大题) "设计一个支持多租户的企业知识库 RAG 系统, 重点讲权限与质量保障。" 参考框架: - 总: 两条管道 + 权限贯穿设计原则; - 分1 架构: pgvector + BM25 双路 → RRF → rerank (top50 封顶) → 接地生成; - 分2 权限: 入库 schema 校验租户/部门字段 → 检索前 where 过滤 (对比检索后过滤两大问题) → 生成端无权限内容零出现; - 分3 质量: RAGAS 四指标定位口诀、评估集含拒答题、版本治理 (单一生效版本)、Lost in the Middle 对抗 (重要内容放开头结尾); - 收尾: 成本三件套 (向量缓存/rerank 封顶/答案缓存) 与演进路线 (先 Naive → 加重排 → 按需 GraphRAG/Agentic)。

20. (大题) "RAG、长上下文、微调怎么选? 给你一个你项目的选型过程。" 参考框架: - 总: 三者改的对象不同——数据/输入/权重, 常组合非互斥; - 分: 决策表七维度 (更新频率/成本曲线/权限/审计/风格/知识类型/频次); - 例: CTRM 例——规则问答走 RAG (权限+新鲜度), 单合同深读直塞长上下文, 合同起草语气微调; 实时数值一律路由业务 API 不进任何方案。

【本节自测】

1. 面试答 RAG 大题的结构模板是什么? 要点: "总—分—例": 先一句话给结论/架构, 再分维度展开, 最后落到具体项目例子 (CTRM)。
2. 被问"RAG 和微调怎么选", 第一句应该说什么? 要点: 先说"两者改的对象不同——RAG 改数据、微调改权重, 常组合而非互斥", 再展开决策表。

- 3. 面试官追问"你的系统幻觉怎么办", 答题顺序? 要点: 先归因 (召回缺失/版本冲突/模型越界三层), 再逐层给解法, 不要直接背"加 rerank"。
- 4. 被质疑"向量库选型为什么不用 Milvus", 怎么回应? 要点: 按规模与运维成本说理 (百万级 pgvector 够用、零新增运维), 并给出升舱触发条件——展示的是决策过程而非站队。
- 5. 答不上来的面试题, 最优策略? 要点: 如实说边界+给推理路径 ("这个我没实际用过, 但类比 X 我会这么推"); 企业场景同理——允许拒答优于编造。

六、得分自评

区间	判定	行动
≥16 题	可出师	去读 003/008 做 Agentic RAG 与工程化串联
10~15	主干牢、细节漏	回错题对应篇目重读
<10	重来一遍	按 README 学习路径走完, 重点 02/03